Project OLED Display

AYA MOHAMED
HABIBA AHMED
BOSINA SAAD
NADIA KAMEL

2024

Name: Aya Mohamed Sayed

Sec:1

Name :Habiba Ahmed Foad

Sec :1

Name :Bosina Saad

Sec :1

Name: Nadia Kamel

Sec :4

I2C OLED Display with Arduino

An OLED display is the best alternative to the character LCD. The OLED display has thin design and high-contrast screen.

OLED Display

Organic Light-Emitting Diode or OLED Display is made with sheets of organic material that emit light in response to an electric current. This sheet is placed between two electrodes and at least one electrode needs to be transparent. So that we could see the light coming from the display.
OLED display comes in different shapes like 128×64, 128×32, 64×64, etc. We will use a 128×64-pixel blue OLED display with an Arduino UNO board.

SSD1306 OLED Driver

There are various display controllers available in the market – SSD1306, SSD1327, SH1106, etc. among them SSD1306 is the most popular. The SSD1306 controller has an internal RAM of 128×64 pixels.

It can communicate using both I2C and SPI protocols. I2C protocol only needs 2 pins to communicate with a microcontroller whereas SPI protocol requires a minimum of 5 to 6 pins. But SPI is faster than I2C. So, you must select between speed and pins according to your project's needs.

Power Requirement for SSD1306 OLED

The SSD1306 OLED driver requires 1.65v to 3.3v whereas the display needs 7v to 15v. To meet these two different power requirements there is an onboard voltage regulator which takes an input voltage from 1.8v and 6v and provides a stable 3.3v output voltage. And a charge pump circuit to obtain a higher voltage for the display. So you don't need any extra power circuitry.

A regular 0.96-inch OLED display requires a **3.3v to 5v** power supply and uses about **20mA** current.

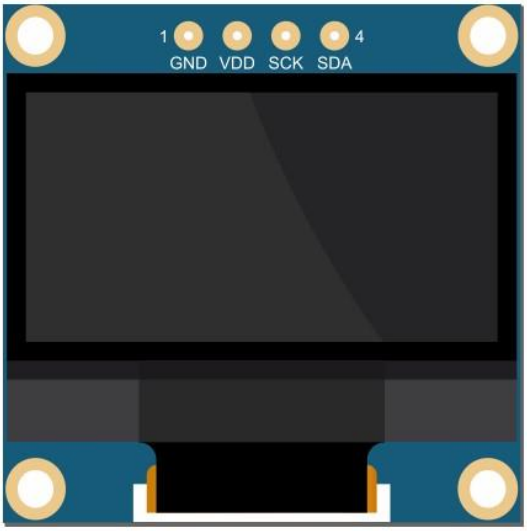
Memory Mapping

The SSD1306 controller has a bit-mapped static RAM called GDDRAM (Graphic Display Data RAM). The size of the RAM is 128 x 64 bits (8,192 bits or 1KB). It holds the bit pattern to be displayed on the screen. This 1KB memory is divided

into eight pages, from PAGE0 to PAGE7, which are used for monochrome 128×64 dot matrix displays. Each page contains 128 columns/segments (blocks 0 to 127). And each column can store 8 bits of data (from 0 to 7).

OLED Display Pinout

Here you can see the picture of an I2C OLED display. It has only four pins. From the picture, you can see that the left two pins are ground and VCC and the other two pins are SCL or SCK, and SDA.

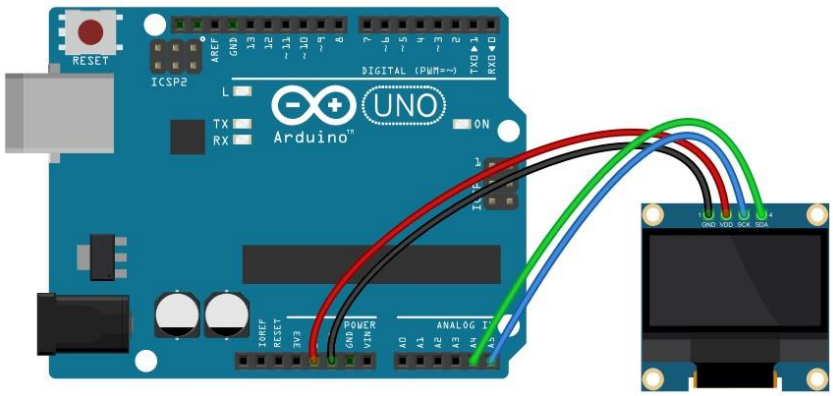


Arduino OLED Connection

An I2C OLED display has 4 pins VSS, Ground, SCL, and SDA. The display needs 3.3v to 5v to operate. So connect the VSS pin to the Arduino 5v pin and ground to the Arduino ground.

If you are using a different Arduino board, check the correct SCL and SDA pin for that board.

- Arduino UNO : SCL > A5, SDA > A4
- Arduino Nano : SCL > A5, SDA > A4
- Arduino Pro Mini : SCL > A5, SDA > A4
- Arduino Mega : SCL > 21, SDA > 20
- Arduino Leonardo : SCL > 21, SDA > 20



Start With OLED

Name: Aya Mohamed Sayed

Sec:1

Name : Bosina Saad

Sec:1

-we need 2 Libraries To control the OLED display and the
1) adafruit_SSD1306.h 2) adafruit_GFX.h libraries.

Common Functions and Their Parameters

1-
```
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
```
This will define a display object with screen width, screen height, and a reset pin for the display. &Wire is used for the I2C communication protocol. OLED_RESET is for the reset pin. If we have a separate reset pin on the OLED connect that pin to an Arduino pin and put that pin number in the place of OLED_RESET But we use OLED with no reset pin to we set it -1.

```
2-display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
```

This will initialize the display with the I2C address 0x3C, and SSD1306_SWITCHCAPVCC will generate a display voltage of 3.3V internally.

Declaration important functions !

```
#include <Wire.h>

#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
```

```
#include <Fonts/FreeSerif9pt7b.h>


#define SCREEN_WIDTH 128 // OLED display width, in pixels
#define SCREEN_HEIGHT 64 // OLED display height, in pixels
#define OLED_RESET     -1

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
```

The Adafruit SSD1306 library has several functions to print text and numbers to the OLED display**, so let's explain this code!**

First, we need to import the necessary library. The Wire library is used for the I2C communication protocol.
Adafruit_SSD1306 library is used to control SSD1306 OLED displays and **Adafruit_GFX** library is used for graphical functionality like drawing points, lines, circles, etc.
Then we need to define OLED height and width. Here we are using a 128×64 pixel so we set the width 128 and height 64.

```
#define SCREEN_WIDTH 128 // OLED display width, in pixels
#define SCREEN_HEIGHT 64 // OLED display height, in pixels
```

Then we create a display object with a previously defined width and height. `&Wire` is used because we are using the I2C communication protocol. The last parameter is for the reset pin.

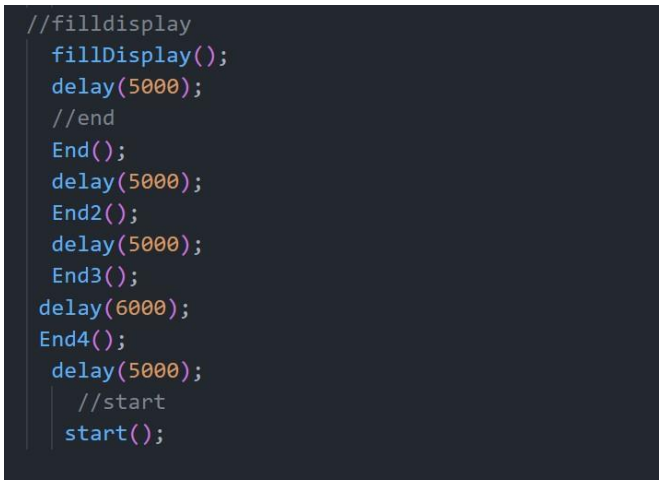
```
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
```

In the setup section, we initialize the display with `begin()` methode. `SSD1306_SWITCHCAPVCC` is used to generate display voltage from 3.3V internally and `0x3C` is the I2C address of the display.

```
  // initialize with the I2C addr 0x3C
display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
```

The SETUP SECTION

```
void setup() {
  // initialize with the I2C addr 0x3C
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  // Clear the buffer
  display.clearDisplay();
  //Hello message
   hellomessage();
   //delay
   delay(5000);
  //scrolling
     Scrolling();
     delay(5000);
  //filldisplay
    fillDisplay();
    delay(5000);
    //end
    End();
```



```
//filldisplay
  fillDisplay();
  delay(5000);
  //end
  End();
  delay(5000);
  End2();
  delay(5000);
  End3();
delay(6000);
End4();
 delay(5000);
   //start
  start();
```

Explain The Code

After initializing the display we use clearDisplay() function to clear The Display Buffer

display.clearDisplay();

Then we Divide our code to make it easy for handling and coding so we have some functions

-hellomessage();
-Scrolling();
-fillDisplay();
-End(); and it has 4 versions!
-start();

Let's Define each function and it's rule

Hellomessage(); Function

```
void hellomessage()
{
  // Display Text
   display.setTextSize(3);
  display.setTextColor(WHITE);
  display.setCursor(15, 0);
  display.print("Hello!");

  display.setTextSize(1);
  display.setTextColor(WHITE);
  display.setCursor(25, 35);
  display.print("Pulse Oximeter ");

   display.setTextSize(1);
  display.setTextColor(BLACK, WHITE);
  display.setCursor(40, 55);
  display.print("Project ");
 display.display();

}
```

Explain The code

Printing Text & Numbers

Set the size of the text using the setTextSize() function. we can set any size starting from 1 but use a suitable size for text to display properly on the screen. We will use text size 3

Then set the text color using the setTextColor() function. WHITE makes the text bright and the background dark

Before printing anything we need to set the cursor position using `setCursor(x,y)` function. Increasing the x value will move the cursor to the right and increasing the y value will move the cursor to the bottom. `print()` will send any text, character, or number to the Display Data RAM (GDDRAM)

```
display.setCursor(15, 0); display.print("Hello!");
```

`display()` will print what is on the Display Data RAM (GDDRAM) or buffer. So a `print()` function alone can't display data to the screen. You need to use the `display()` function after `print()` to see the actual data on the screen.

```
display.display();
```

.and The same for display the message 'Pulse Oximeter' &'project'

With Different locations on the screen

Scrolling(); Function

Scrolling Text

The Adafruit OLED library provides useful methods to easily scroll text.

- `startscrollright(0x00, 0x0F)`: scroll text from left to right
- `startscrollleft(0x00, 0x0F)`: scroll text from right to left
- `startscrolldiagright(0x00, 0x07)`: scroll text from left bottom corner to right upper corner
- `startscrolldiagleft(0x00, 0x07)`: scroll text from right bottom corner to left upper corner

```
void Scrolling()
{
    display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(WHITE);
  display.print("ByOLED");

  display.display();

  display.startscrolldiagright(0, 7);
  delay(5000);
  display.stopscroll();
  display.startscrolldiagleft(0, 7);
  delay(5000);
  display.stopscroll();
}
```

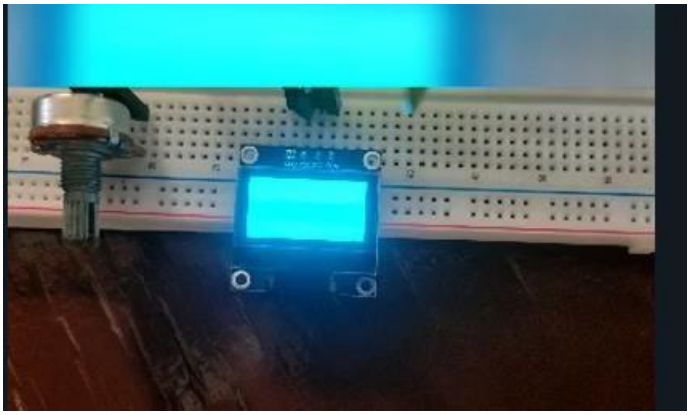
so after clear the display and set text size and color and print it we have scrolled it by function display.startscrolldiagright(0,7) the x and y position and beween every scroll we have a delay of 5 s and we can stop scrolling by dunction

```
display.startscrolldiagright(0, 7);
delay(5000);
display.stopscroll();
display.startscrolldiagleft(0, 7);
delay(5000);
display.stopscroll();
```

filldisplay(); Function

```
void fillDisplay(void) {

  display.clearDisplay();
  for (int j = 63; j >= 0; j--) {
    for (int i = 127; i >= 0; i--) {
      display.drawPixel(i, j, WHITE);
    }
    display.display();
    delay(2);
  }
}
```

It will fill all the pixels of your OLED line by line



End Functions and its Versions

Name :Nadia Kamel

Sec :4

Name: Aya Mohamed Sayed

Sec :1

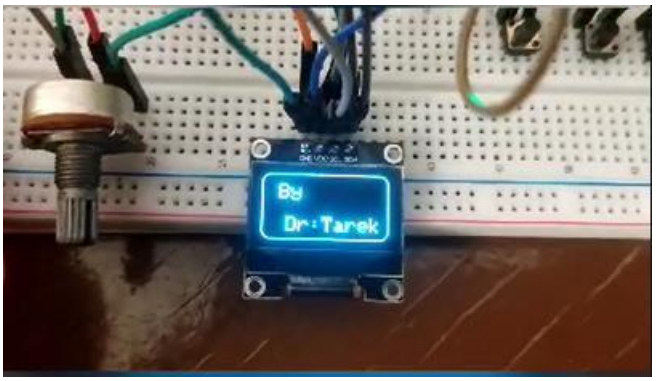
```
void End()
{

 display.clearDisplay();
display.drawRoundRect(0, 0, 127, 64, 8, WHITE);
display.setTextSize(2);
display.setTextColor(WHITE);
display.setCursor(5, 10);

display.print(" By\n\n  Dr:Tarek ");

display.display();

}
```

```
void End2()
{
    display.clearDisplay();
display.drawRoundRect(0, 0, 127, 64, 8, WHITE);
display.setTextSize(1);
display.setTextColor(WHITE);
display.setCursor(0, 10);

display.print("      Communication \n   &  \n     electronincs\n\n       Department");

display.display();
 delay(5000);
   display.invertDisplay(true);
  delay(2000);
  display.invertDisplay(false);
  delay(2000);
}
```

```
void End3()
{
    display.clearDisplay();
display.drawRoundRect(0, 0, 127, 64, 8, WHITE);
display.setTextSize(1);
display.setTextColor(WHITE);
display.setCursor(0, 10);
display.print("  AyaMohamed   \n");
display.print("\n        HabibaAhmed  \n ");
display.setCursor(0, 30);
display.print("\n  BosinaSaad   \n");
display.print("\n        NadiaKamal  \n ");
display.setCursor(35, 25);
  display.write(2);
 display.setCursor(80, 40);
  display.write(2);
display.display();

}
```

```
void End4()
{
    display.drawBitmap(0, 0,  shoubra, 128, 64, WHITE);
  display.display();
}
```

all end function will display the team project and Department and Bitmap of LOGO of Shubra Faculty of Engineering



start();Function

```
void start()
{
 display.setFont(&FreeSerif9pt7b);
 display.clearDisplay();
display.drawRoundRect(0, 0, 127, 64, 8, WHITE);
display.setTextSize(1);
display.setTextColor(WHITE);
display.setCursor(5, 26);
display.print("        Enter\n        Finger ");
display.display();
}
```

Then we will use this function to start our application on OLED this app is PULSE-Oximeter project we start this function with statement enter finger But we change the type of text with function. But we must use this library

```
#include <Fonts/FreeSerif9pt7b.h>
```

```
display.setFont(&FreeSerif9pt7b);
```

Animation :

Name: Bosina Saad

Sec :1

Explain Animation code

```
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_I2C_ADDR 0x3c // or 0x3C
#define SCREEN_WIDTH 128     // OLED display width, in pixels
#define SCREEN_HEIGHT 64     // OLED display height, in pixels
#define OLED_RST_PIN -1      // Reset pin (-1 if not available)

Adafruit_SSD1306 display(128, 64, &Wire, OLED_RST_PIN);

// OLED Animation: ok hand


#define FRAME_DELAY (42)
#define FRAME_WIDTH (64)
#define FRAME_HEIGHT (64)
#define FRAME_COUNT (sizeof(frames) / sizeof(frames[0]))
```

This code is setting up an environment to work with an SSD1306-based OLED display using the Adafruit GFX and SSD1306 libraries. Let's break it down step by step:

1. `#include <Adafruit_GFX.h>` and `#include <Adafruit_SSD1306.h>`: These lines include the necessary header files to work with the Adafruit GFX graphics library and the Adafruit SSD1306 OLED display driver library.

2. `#define SCREEN_I2C_ADDR 0x3c`: This line defines the I2C address of the OLED display. The value `0x3C` is commonly used for SSD1306 displays.

3. `#define SCREEN_WIDTH 128` and `#define SCREEN_HEIGHT 64`: These lines define the width and height of the OLED display in pixels. The display is 128 pixels wide and 64 pixels tall.

4. `#define OLED_RST_PIN -1`: This line defines the reset pin for the OLED display. The value `-1` indicates that there is no reset pin connected. If you have a reset pin available and connected, you would specify its pin number here.

5. `Adafruit_SSD1306 display(128, 64, &Wire, OLED_RST_PIN);`: This line initializes an instance of the `Adafruit_SSD1306` class named `display`. It specifies the width and height of the display (128x64 pixels), the I2C interface (`&Wire`), and the reset pin (`OLED_RST_PIN`). This prepares the OLED display for use.

6. `#define FRAME_DELAY (42)`: This line defines the delay between frames in an animation. The value `42` represents the delay in milliseconds.

7. `#define FRAME_WIDTH (64)` and `#define FRAME_HEIGHT (64)`: These lines define the width and height of each frame in an animation. Each frame is a square with dimensions 64x64 pixels.

8. `#define FRAME_COUNT (sizeof(frames) / sizeof(frames[0]))`: This line calculates the number of frames in the animation. It divides the total size of the `frames` array by the size of a single frame element (`frames[0]`). However, the code snippet you provided does not include the definition or initialization of the `frames` array, so it's not clear how this would be used without further context.

In summary, this code sets up the necessary parameters and initializes an SSD1306-based OLED display for use with the Arduino platform. It also includes definitions for parameters related to an animation, but the animation itself is not provided in this snippet.

```
const byte PROGMEM frames[][512] = {
  {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,248,0,0,0,0,0,0,1,252,0,0,0,0,0,0,
  {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,248,0,0,0,0,0,0,1,252,0,0,0,0,0,0,
  {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,248,0,0,0,0,0,0,1,252,0,0,0,0,0,0,
  {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,248,0,0,0,0,0,0,1,252,0,0,0,0,0,0,
  {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,240,0,0,0,0,0,0,3,248,0,0,0,0,0,0,
  {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,224,0,0,0,0,0,0,7,240,0,0,0,0,0,0,
  {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,62,0,0,0,0,0,0,0,127,0,0,0,0,0,0,0
  {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,28,0,0,0,0,0,0,0,63,0,0,0,0,0,0,0,127,0,0,0,0,0,
  {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,192,31,0,0,0,0,0,3,240,63,128,0,
  {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,62,0,0,0,0,0,0,0,127,1,240,0,0,0,0,0,119,131,248
```

The code you provided initializes a two-dimensional array named frames in PROGMEM (Program Memory) containing sequences of bytes. Let's break down this initialization:

1. const byte PROGMEM frames[][512]: This line declares a two-dimensional array named frames. It's declared as const, meaning its contents cannot be modified during runtime. The PROGMEM keyword indicates that this array should be stored in program memory, which is a type of read-only memory used in Arduino devices to store data that doesn't change during program execution. Each element of the array is of type byte, indicating it stores a single byte of data. The [][512] syntax means frames is an array of arrays, and each inner array has 512 elements.

2.{0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,248,0,0,0,0,0,0,1,252,0,0,0,0,0,0,0,3,158,0,0,0,0,0,0,3,15,0,0,0,0,0,0,3,7,0,0,0,0,0,0,......}: This part initializes the contents of the frames array. Each element of frames is an array of 512 bytes. The provided sequence of bytes represents an image or frame in a bitmap format. Each byte likely corresponds to the brightness or color of a pixel in the image, with 0 representing black (off) and non-zero values representing various shades of white or color.

```
void setup() {
  display.begin(SSD1306_SWITCHCAPVCC, SCREEN_I2C_ADDR);
}

int frame = 0;
void loop() {
  display.clearDisplay();
  display.drawBitmap(32, 0, frames[frame], FRAME_WIDTH, FRAME_HEIGHT, 1);
  display.display();
  frame = (frame + 1) % FRAME_COUNT;
  delay(FRAME_DELAY);
}
```

This code sets up an Arduino sketch to display frames of an animation on an SSD1306-based OLED display. Let's go through it step by step:

1. void setup() {...}: This is the setup function, called once when the microcontroller starts. In this function:

- display.begin(SSD1306_SWITCHCAPVCC, SCREEN_I2C_ADDR);: It initializes the OLED display. SSD1306_SWITCHCAPVCC is a power supply type, and SCREEN_I2C_ADDR is the I2C address of the display. This function configures the display to start communicating over I2C.

2. int frame = 0;: This declares an integer variable frame and initializes it to 0. This variable is used to keep track of the current frame of the animation.

3. void loop() {...}: This is the main loop function, repeatedly executed by the microcontroller.

- display.clearDisplay();: Clears the display buffer.

- display.drawBitmap(32, 0, frames[frame], FRAME_WIDTH, FRAME_HEIGHT, 1);: Draws a bitmap onto the display buffer. It takes parameters for the position (32, 0) where the bitmap will start, the bitmap data from the frames array for the current frame, the width and height of the frame, and the color (1 for white, 0 for black).

- display.display();: Renders the contents of the display buffer onto the OLED display.

- frame = (frame + 1) % FRAME_COUNT;: Updates the frame variable to the next frame. It wraps around to 0 when it reaches the end of the animation.

- delay(FRAME_DELAY);: Pauses execution for a specified period of time (42 milliseconds in this case), controlling the speed of the animation.

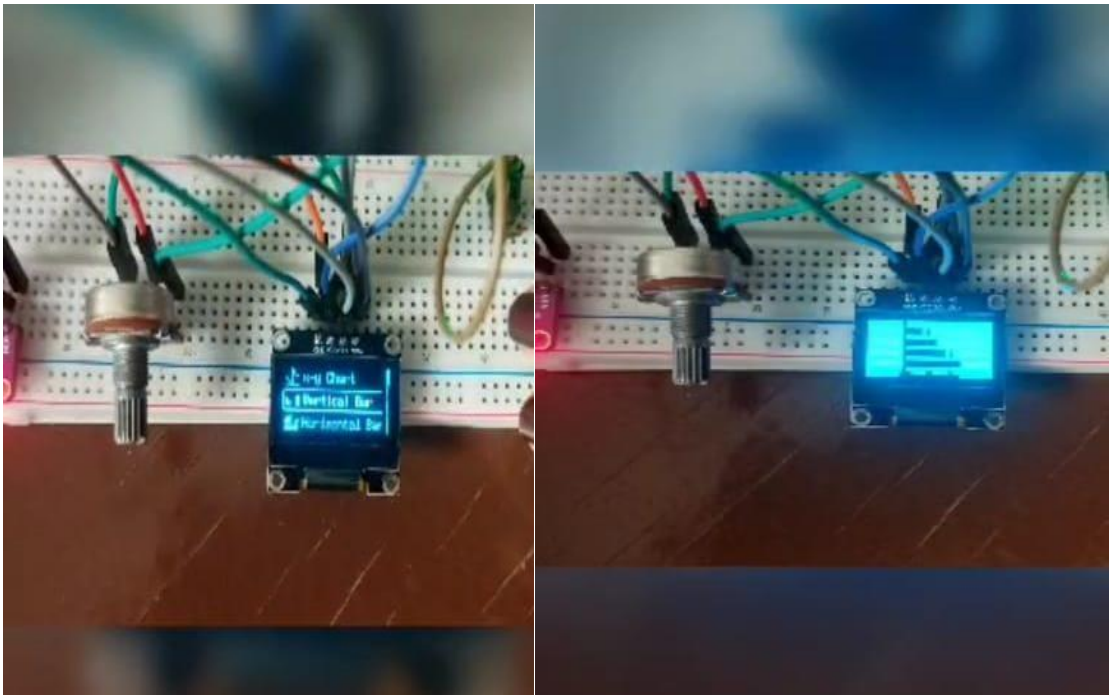
This loop continuously cycles through the frames of the animation, displaying each frame on the OLED display. When it reaches the end of the animation, it starts over from the beginning.

Menu

Name: Habiba Ahmed Foad

Sec :1

In this project we display 4 items of images of different types of graphs, So we make Bitmap using image2cpp site.



```
#include "U8g2lib.h"

U8G2_SSD1306_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0); // [full framebuffer, size = 1024 bytes]
```

Using U8g2 is monochrome graphics library. contains both drivers and high-level drawing routines. Then we make object from it for using.

In main screen which we display item in, we display icon 16 x 16 px for each item using bitmap.

Then make array for pointer to each icon array.

```
const unsigned char bitmap_barv [] PROGMEM = {
  0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x01, 0xc0, 0x01, 0xf0, 0x01, 0xf0, 0x07, 0xf0,
  0x07, 0xf0, 0x37, 0xf0, 0x37, 0xf0, 0x37, 0xf0, 0x37, 0xf0, 0x36, 0xf0, 0x00, 0x00, 0x00, 0x00};


const unsigned char bitmap_barh [] PROGMEM = {
  0x80, 0x00, 0x80, 0x00, 0xbe, 0x00, 0xbe, 0x00, 0x80, 0x00, 0xbf, 0xc0, 0xbf, 0xc0, 0x80, 0x00,
  0xbf, 0xf8, 0xbf, 0xf8, 0x80, 0x00, 0xbf, 0xff, 0xbf, 0xff, 0x80, 0x00, 0x92, 0x48, 0xff, 0xff};
const unsigned char bitmap_dial [] PROGMEM = {
  0xE0, 0x07, 0x18, 0x18, 0x84, 0x24, 0x0A, 0x40, 0x12, 0x50, 0x21, 0x80,
  0xC1, 0x81, 0x45, 0xA2, 0x41, 0x82, 0x81, 0x81, 0x05, 0xA0, 0x02, 0x40,
  0xD2, 0x4B, 0xC4, 0x23, 0x18, 0x18, 0xE0, 0x07, };
const unsigned char bitmap_chart [] PROGMEM = {
  0x80, 0x01, 0x80, 0x00, 0x80, 0x02, 0x80, 0x04, 0x80, 0x04, 0x80, 0x08, 0x80, 0xc8, 0x80, 0x30,
  0x81, 0x00, 0x82, 0x00, 0x80, 0x00, 0x9c, 0x00, 0x80, 0x00, 0xa0, 0x00, 0x80, 0x00, 0xff, 0xff};



const unsigned char* bitmap_icons[4] = {
  bitmap_barv,
  bitmap_barh,
  bitmap_dial,
  bitmap_chart,
};
```

The next screen that appears when we click the select button is images of graphs so we have to make them also as bitmaps and make array pointer to each one

```
// images 128x64
const unsigned char bitmap_Barv [] PROGMEM = {
  0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xfc, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xfc, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xfd, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xfd, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xfd, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xfd, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xfd, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xfd, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xfd, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xfd, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
```

```
const unsigned char* bitmap_images[4] = {
  bitmap_Barv,
  bitmap_Barh,
  bitmap_Dial,
  bitmap_Chart,
};
```

In menu screen we display also scrollbar and outline for item selected using also bitmaps

```
//8x64px
const unsigned char bitmap_scrollbar [] PROGMEM = {
  0x00, 0x40, 0x00, 0x40, 0x00, 0x40, 0x00, 0x40, 0x00, 0x40, 0x00, 0x40,
  0x00, 0x40, 0x00, 0x40, 0x00, 0x40, 0x00, 0x40, 0x00, 0x40, 0x00, 0x40,
  0x00, 0x40, 0x00, 0x40, 0x00, 0x40, 0x00, 0x40, 0x00, 0x40, 0x00, 0x40,
  0x00, 0x40, 0x00, 0x40, 0x00, 0x40, 0x00, 0x40, 0x00, 0x40, 0x00, 0x40,
  0x00, 0x40, 0x00, 0x40, 0x00, 0x40, 0x00, 0x40, 0x00, 0x40, 0x00, 0x40,
  0x00, 0x40, 0x00, 0x00, };


//128x21p
const unsigned char bitmap_item_sel_outline [] PROGMEM = {
  0xF8, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF,
  0xFF, 0xFF, 0xFF, 0x03, 0x04, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
  0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x04, 0x02, 0x00, 0x00, 0x00,
  0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0C,
  0x02, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
  0x00, 0x00, 0x00, 0x0C, 0x02, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
  0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0C, 0x02, 0x00, 0x00, 0x00,
  0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0C,
  0x02, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
  0x00, 0x00, 0x00, 0x0C, 0x02, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
  0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0C, 0x02, 0x00, 0x00, 0x00,
```

Text of items name stored in array .

```
const int NUM_ITEMS = 4;
 const int MAX_ITEM_LENGTH = 20;
//item name
char menu_items [NUM_ITEMS] [MAX_ITEM_LENGTH] = {
  { "Vertical Bar" },
  { "Horizontal Bar" },
  { "Dial" },
  { "x-y Chart" },
 };
```

Define the Button-up for the previous item, button down for the next item and the select button for the select item. And another some variables that we will need.

```
#define BUTTON_UP_PIN 12
#define BUTTON_SELECT_PIN 8
#define BUTTON_DOWN_PIN 4

int button_up_clicked = 0;
int button_select_clicked = 0;
int button_down_clicked = 0;

int item_selected = 0;

int item_sel_previous;
int item_sel_next;

int current_screen = 0;   // 0 = menu, 1 = images
```

Begin to store the data in the buffer of U8g2lib.

```
void setup() {
  u8g2.begin();
```

Define the mode of each pin.

```
  // INPUT_PULLUP means the button is HIGH when not pressed, and LOW when pressed
  pinMode(BUTTON_UP_PIN, INPUT_PULLUP);
  pinMode(BUTTON_SELECT_PIN, INPUT_PULLUP);
  pinMode(BUTTON_DOWN_PIN, INPUT_PULLUP);
}
```

We defined the current screen to determined which screen that will be displayed. And controlled of up-down buttons to work only in menu screen (current screen =0) Curren screen value changed when the select button cliched to display the image.

```
void loop() {
 if (current_screen == 0) {
     // up and down buttons only work for the menu screen
     if ((digitalRead(BUTTON_UP_PIN) == LOW) && (button_up_clicked == 0)) {
       item_selected = item_selected - 1;
       button_up_clicked = 1;
       if (item_selected < 0) {
         item_selected = NUM_ITEMS-1;
       }
     }
     else if ((digitalRead(BUTTON_DOWN_PIN) == LOW) && (button_down_clicked == 0)) {
       item_selected = item_selected + 1;
       button_down_clicked = 1;
       if (item_selected >= NUM_ITEMS) {
         item_selected = 0;
         }
     }

     if ((digitalRead(BUTTON_UP_PIN) == HIGH) && (button_up_clicked == 1)) {
       button_up_clicked = 0;
     }
     if ((digitalRead(BUTTON_DOWN_PIN) == HIGH) && (button_down_clicked == 1)) {
       button_down_clicked = 0;
     }

 }


 if ((digitalRead(BUTTON_SELECT_PIN) == LOW) && (button_select_clicked == 0)) {
    button_select_clicked = 1;
    if (current_screen == 0) {current_screen = 1;}
    else {current_screen = 0;}
 }
 if ((digitalRead(BUTTON_SELECT_PIN) == HIGH) && (button_select_clicked == 1)) {
   button_select_clicked = 0;
 }
```

Changes the value of item select previous and next and selected after buttons clicked.

```
    item_sel_previous = item_selected - 1;
    if (item_sel_previous < 0) {item_sel_previous = NUM_ITEMS - 1;}
    item_sel_next = item_selected + 1;
    if (item_sel_next >= NUM_ITEMS) {item_sel_next = 0;}
```

Now clear buffer from any previous display.

```
u8g2.clearBuffer();
```

Check which screen will be displayed and use some function to display it's element.

```
if (current_screen == 0) {

  u8g2.drawXBMP(0, 22, 128, 21, bitmap_item_sel_outline);

  // draw previous
  u8g2.setFont(u8g_font_7x14);
  u8g2.drawStr(25, 15, menu_items[item_sel_previous]);
  u8g2.drawXBMP( 4, 2, 16, 16, bitmap_icons[item_sel_previous]);

  // draw selected
  u8g2.setFont(u8g_font_7x14B);
  u8g2.drawStr(25, 15+20+2, menu_items[item_selected]);
  u8g2.drawXBMP( 4, 24, 16, 16, bitmap_icons[item_selected]);

  // draw next
  u8g2.setFont(u8g_font_7x14);
  u8g2.drawStr(25, 15+20+20+2+2, menu_items[item_sel_next]);
  u8g2.drawXBMP( 4, 46, 16, 16, bitmap_icons[item_sel_next]);

  // draw scrollbar
  u8g2.drawXBMP(128-8, 0, 8, 64, bitmap_scrollbar);

  // draw scrollbar
  u8g2.drawBox(125, 64/NUM_ITEMS * item_selected, 3, 64/NUM_ITEMS);

}
```

Explanation of each function:

u8g2.setFont();❾ For change font

u8g2.drawStr(x, y, item]);❾ For displaying string.

u8g2.drawXBMP(x, y, w, h, bitmap item); ❾ for display bitmaps

u8g2.setColorIndex();❾ for Change color 1❾white ,0❾black

u8g2.sendBuffer(); ❾ send buffer from RAM to display.

Note that the order of icon in its array must be the order of image array because we pass the item select as the index for each on.

```
//images
    else if (current_screen == 1) {
        // u8g2.setColorIndex(0);
        //draw images
        u8g2.drawXBMP( 0, 0, 128, 64, bitmap_images[item_selected]);
        // u8g2.setColorIndex(1);

    }
u8g2.sendBuffer(); // send buffer from RAM to display
```

OLED For Graph Display

Name: Aya Mohamed Sayed

Sec:1

We will display 4 Graphs:

1. horizontal bar graph
2. vertical bar graph
3. dial
4. cartesian graph

Let's Start our code & explain it

```
#include<Adafruit_GFX.h>
#include<Adafruit_SSD1306.h>

#define READ_PIN      A0
#define OLED_RESET    -1
#define LOGO16_GLCD_HEIGHT 16
#define LOGO16_GLCD_WIDTH  16
```

Explaining the code

-First, you need to import the necessary library. The Wire library is used for the I2C communication protocol. If you use an SPI OLED display use the SPI library. Adafruit_SSD1306 library is used to control SSD1306 OLED displays and Adafruit_GFX library is used for graphical functionality like drawing points, lines, circles, etc.

-Second we define the analog input from POT. To pin A0

-Third OLED reset to -1
If our display has a separate reset pin connect that pin to the Arduino and use that pin number here. Otherwise use -1 to use the Arduino reset pin to reset the display. So we put -1

Then we need to define LOGO height and width to 16

Horizontal Graph Display

Calling Function & its Parameter & its Function

```
//FOR Calling Function
  DrawBarChartH(display, volts, 10, 45, 100, 20, 0, 5, 1, 0, "A0 (volts)",
Redraw2);
```

The OUTPUT OF THIS FUNCTION

Explain The Function

The `DrawBarChartH` function is designed to create a **horizontal bar chart** on a specified display. Let's break down its purpose and the meaning of its arguments:

1. **`display`**: The display object on which the horizontal bar chart will be drawn.
2. **`data`**: A list of data values for the bars.
3. **`x`**: The x-coordinate of the top-left corner of the chart.
4. **`y`**: The y-coordinate of the top-left corner of the chart.
5. **`width`**: The width of the chart.
6. **`height`**: The height of the chart.
7. **`min_value`**: The minimum value for the data.
8. **`max_value`**: The maximum value for the data.
9. **`num_bars`**: The number of bars to display.
10. **`bar_spacing`**: The spacing between bars.
11. **`label`**: The label for the x-axis.
12. **`redraw_callback`**: A callback function to redraw the display after changes.

The implementation details of this function would involve:

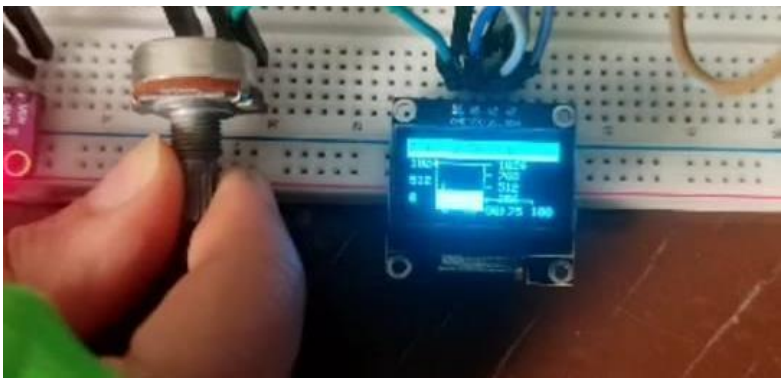
- Calculating the bar height based on the available height and the number of bars.
- Normalizing the data values to fit within the chart width.
- Drawing the bars and labels on the display.
- Calling the `redraw_callback` to update the display.

Vertical Graph Display

Calling Function & its Parameter & its Function

```
//Calling Function
DrawBarChartV(display, bvolts, 25,  60, 40, 40, 0, 1024 , 256, 0, "A0 (bits)",
Redraw1);
```

The OUTPUT OF THIS FUNCTION



Explain The Function

The function `DrawBarChartV` is designed to create a vertical bar chart on a display. Let's break down its purpose and the meaning of its arguments:

1. **`display`**: The display object on which the bar chart will be drawn.
2. **`data`**: A list of data values for the bars.
3. **`x`**: The x-coordinate of the top-left corner of the chart.
4. **`y`**: The y-coordinate of the top-left corner of the chart.
5. **`width`**: The width of the chart.
6. **`height`**: The height of the chart.
7. **`min_value`**: The minimum value for the data.
8. **`max_value`**: The maximum value for the data.
9. **`num_bars`**: The number of bars to display.
10. **`bar_spacing`**: The spacing between bars.
11. **`label`**: The label for the x-axis.
12. **`redraw_callback`**: A callback function to redraw the display after changes.

The implementation details of this function would involve:

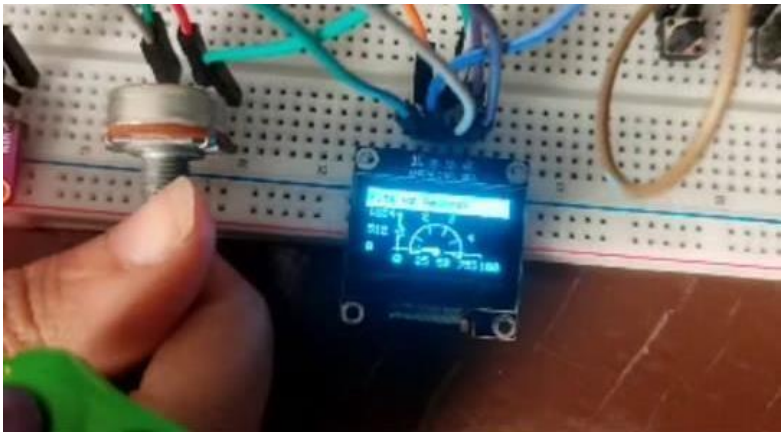
- Calculating the bar width based on the available width and the number of bars.
- Normalizing the data values to fit within the chart height.
- Drawing the bars and labels on the display.
- Calling the `redraw_callback` to update the display.

DrawDail Graph Display

Calling Function & its Parameter & its Function

```
//Calling Function
 DrawDial(display, volts, 65, 50, 25, 0, 5 , 1, 0, 200, "A0 (volts)", Redraw3);
```

The OUTPUT OF THIS FUNCTION



Explain The Function

The `DrawDial` function is designed to create a **circular dial (or gauge)** display on a specified screen. Let's break down its purpose and the meaning of its arguments:

1. **`display`**: The display object on which the dial will be drawn.
2. **`data`**: The value (in volts) to be displayed by the dial.
3. **`x`**: The x-coordinate of the center of the dial.
4. **`y`**: The y-coordinate of the center of the dial.
5. **`radius`**: The radius of the dial (size of the circular display).
6. **`min_value`**: The minimum value for the data (e.g., 0 volts).
7. **`max_value`**: The maximum value for the data (e.g., 5 volts).
8. **`step`**: The step size (increment) for the dial (e.g., 1 volt).
9. **`start_angle`**: The starting angle (in degrees) for the dial (e.g., 0 degrees).
10. **`end_angle`**: The ending angle (in degrees) for the dial (e.g., 200 degrees).
11. **`label`**: The label for the dial (e.g., "A0 (volts)").
12. **`redraw_callback`**: A callback function to redraw the display after changes.

The implementation details of this function would involve:

- Calculating the position of the needle based on the data value.
- Drawing the circular dial with markings.
- Drawing the needle pointing to the data value.
- Displaying the label.

MultiGraph Display

```
  // for multiple graphs
    DrawBarChartV(display, bvolts, 5,  60, 10, 40, 0, 1024 , 256, 0, "A0
(bits/volts)", Redraw1);
    DrawDial(display, volts, 90, 50, 25, 0, 5 , 1, 0, 200, "A0 (bits/volts)",
```

```
Redraw3);
```

The OUTPUT OF THIS FUNCTION

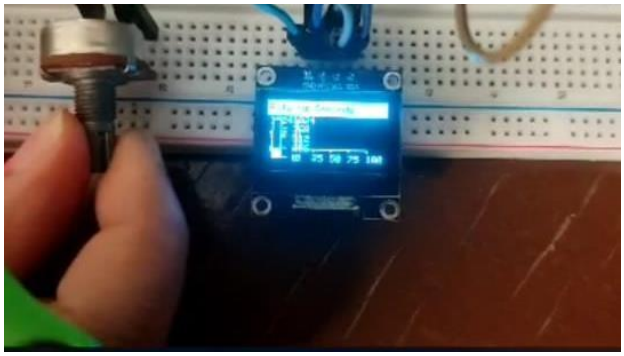
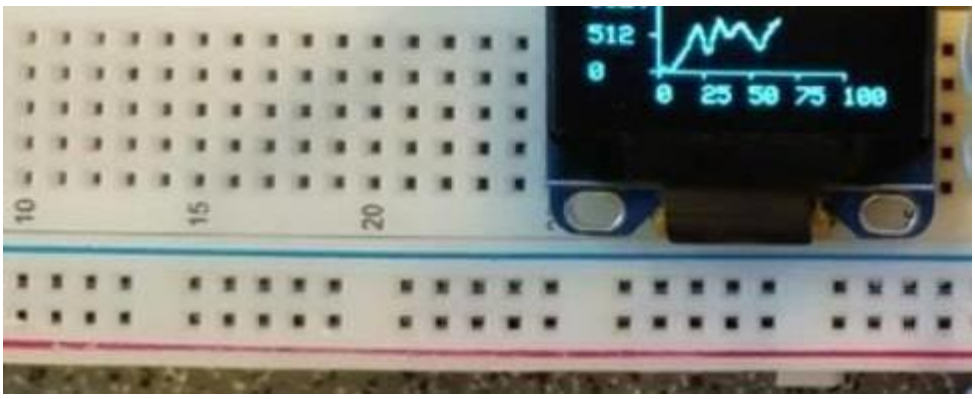


Chart Display

Calling Function & its Parameter & its Function

```
// or show a cartesian style graph to plot values over time (or whatever)
//Calling Function
    DrawCGraph(display, x++, bvolts, 30, 50, 75, 30, 0, 100, 25, 0, 1024, 512, 0,
"Bits vs Seconds", Redraw4);
```

The OUTPUT OF THIS FUNCTION



Explain The Function

The `DrawCGraph` function is designed to create a continuous graph (line chart) on a specified display. Let's break down its purpose and the meaning of its arguments:

1. **`display`**: The display object on which the graph will be drawn.
2. **`x`**: The current x-coordinate (used for plotting the next data point).
3. **`data`**: A list of y-values (data points) to be plotted.
4. **`x`**: The x-coordinate of the top-left corner of the graph.
5. **`y`**: The y-coordinate of the top-left corner of the graph.
6. **`width`**: The width of the graph.
7. **`height`**: The height of the graph.
8. **`min_x`**: The minimum value for the x-axis.
9. **`max_x`**: The maximum value for the x-axis.
10. **`step_x`**: The step size (increment) for the x-axis.
11. **`min_y`**: The minimum value for the y-axis.
12. **`max_y`**: The maximum value for the y-axis.
13. **`step_y`**: The step size (increment) for the y-axis. 14. **`label`**: The label for the graph (e.g., "Bits vs Seconds").
15. **`redraw_callback`**: A callback function to redraw the display after changes.

The implementation details of this function would involve:

- Calculating the position of each data point (x, y) within the graph.
- Connecting adjacent data points with lines.
- Drawing axis labels, ticks, and grid lines.
- Calling the `redraw_callback` to update the display.

Blood Oxygen & Heart Rate Monitor

Name: Habiba Ahmed Foad

Sec:1

Introduction

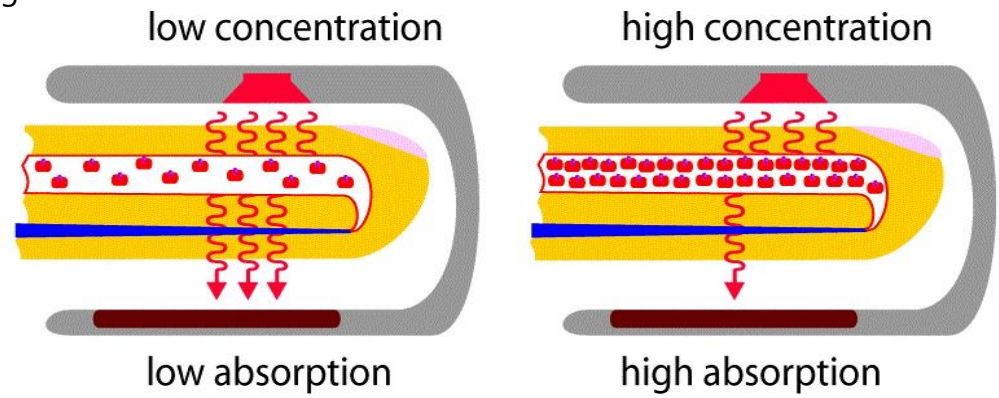
In this project we will make a device that can measure Blood Oxygen & Heart Rate • The blood Oxygen Concentration termed as SpO2 is measured in Percentage and • Heartbeat/Pulse Rate is measured in BPM. We will display the SpO2 and BPM value. With each beat, the display value is changed. •

Components

Arduino uno Board •
Max30100 pulse oximeter sensor •
0.96'' i2c display •
Connecting wires •
Breadboard •

How does Pulse Oximeter Works?

- Oxygen enters the lungs and then is passed on into blood. The blood carries oxygen to the various organs in our body. The main way oxygen is carried in our blood is by means of haemoglobin. During a pulse oximetry reading, a small clamp-like device is placed on a finger, earlobe, or toe.
- small beams of light pass through the blood in the finger, measuring the amount of oxygen. It does this by measuring changes in light absorption in oxygenated or deoxygenated blood.



MAX30100 Pulse Oximeter

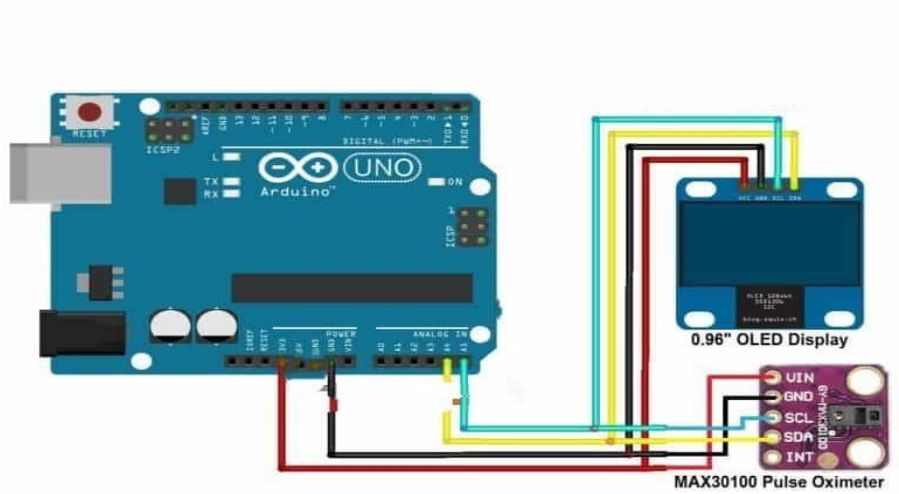
- The sensor is an integrated pulse oximetry and heart rate monitor sensor solution. It combines two LED's, a photo detector, optimized optics, and lownoise analog signal processing to detect pulse and heart-rate signals. It operates from 1.8V and 3.3V power supplies and can be powered down through software with negligible standby current, permitting the power supply to always remain connected.

Working of MAX30100 Pulse Oximeter and Heart-Rate Sensor

- The device has two LEDs, one emitting red light, another emitting infrared light. For pulse rate, only the infrared light is needed. Both the red light and infrared light is used to measure oxygen levels in the blood.
- When the heart pumps blood, there is an increase in oxygenated blood as a result of having more blood. As the heart relaxes, the volume of oxygenated blood also decreases. By knowing the time between the increase and decrease of oxygenated blood, the pulse rate is determined.
- It turns out, oxygenated blood absorbs more infrared light and passes more red light while deoxygenated blood absorbs red light and passes more infrared

light. This is the main function of the MAX30100: it reads the absorption levels for both light sources and stored them in a buffer that can be read via I2C.

Circuit Diagram & Connections .



Explain Code

Name: Nadia Kamel

Sec:4

```
#include <Wire.h>
#include "MAX30100_PulseOximeter.h"

#include "Wire.h"
#include "Adafruit_GFX.h"
#include "OakOLED.h"
```

- Includes necessary libraries for I2C communication (Wire.h), controlling the pulse oximeter (MAX30100_PulseOximeter.h), and

interfacing with the OLED display (Adafruit_GFX.h and OakOLED.h).

```
#define REPORTING_PERIOD_MS 1000
OakOLED oled;

PulseOximeter pox;

uint32_t tsLastReport = 0;

const unsigned char bitmap [] PROGMEM=
{
0x00, 0x00, 0x00, 0x00, 0x01, 0x80, 0x18, 0x00, 0x0f, 0xe0, 0x7f, 0x00, 0x3f, 0xf9, 0xff, 0xc0,
0x7f, 0xf9, 0xff, 0xc0, 0x7f, 0xff, 0xff, 0xe0, 0x7f, 0xff, 0xff, 0xe0, 0xff, 0xff, 0xff, 0xf0,
0xff, 0xf7, 0xff, 0xf0, 0xff, 0xe7, 0xff, 0xf0, 0xff, 0xe7, 0xff, 0xf0, 0x7f, 0xdb, 0xff, 0xe0,
0x7f, 0x9b, 0xff, 0xe0, 0x00, 0x3b, 0xc0, 0x00, 0x3f, 0xf9, 0x9f, 0xc0, 0x3f, 0xfd, 0xbf, 0xc0,
0x1f, 0xfd, 0xbf, 0x80, 0x0f, 0xfd, 0x7f, 0x00, 0x07, 0xfe, 0x7e, 0x00, 0x03, 0xfe, 0xfc, 0x00,
0x01, 0xff, 0xf8, 0x00, 0x00, 0xff, 0xf0, 0x00, 0x00, 0x7f, 0xe0, 0x00, 0x00, 0x3f, 0xc0, 0x00,
0x00, 0x0f, 0x00, 0x00, 0x00, 0x06, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00
};
```

- 1. Defines a reporting period (REPORTING_PERIOD_MS) as 1000 milliseconds, which is 1 second.
- 2. Declares an instance of the OakOLED class named oled.
- 3. Declares an instance of the PulseOximeter class named pox.
- 4. Defines a global variable tsLastReport to keep track of the last time a report was sent.
- 5. Defines a bitmap stored in PROGMEM, likely representing an icon to display on the OLED when a beat is detected. •

```
void onBeatDetected()
{
Serial.println("Beat!");
oled.drawBitmap( 60, 20, bitmap, 28, 28, 1);
oled.display();
}
```

- 1. Defines a function onBeatDetected() to be called whenever a beat is detected. It prints "Beat!" to the serial monitor and displays a bitmap on the OLED.
- 2. oled.drawBitmap(60, 20, bitmap, 28, 28, 1);: This line draws a bitmap on an OLED display. It likely uses a function called drawBitmap() provided by a library for controlling OLED displays. The parameters passed to drawBitmap() are as follows:
- - 60 and 20 are the X and Y coordinates where the bitmap will be drawn on the display.
- - bitmap is the array containing the bitmap data.
- - 28 and 28 are the width and height of the bitmap (in pixels).
- - 1 is a scale factor or color depth. In this case, it's likely indicating a monochrome bitmap (black and white).
- 3. oled.display();: This line updates the OLED display to reflect the changes made by drawing the bitmap. It's likely that oled is

```
void setup()
{
Serial.begin(9600);

oled.begin();
oled.clearDisplay();
oled.setTextSize(1);
oled.setTextColor(1);
oled.setCursor(0, 0);

oled.println("Initializing pulse oximeter..");
oled.display();
Serial.print("Initializing pulse oximeter..");

if (!pox.begin()) {
Serial.println("FAILED");
oled.clearDisplay();
oled.setTextSize(1);
oled.setTextColor(1);
oled.setCursor(0, 0);
oled.println("FAILED");
oled.display();
for(;;);
} else {
oled.clearDisplay();
oled.setTextSize(1);
oled.setTextColor(1);
oled.setCursor(0, 0);
oled.println("SUCCESS");
oled.display();
Serial.println("SUCCESS");


}
pox.setOnBeatDetectedCallback(onBeatDetected);

}
```

an instance of a class representing the OLED display and display() is a method to update the display.

-
-

-
- This setup() function initializes the system. Here's a breakdown of what it does:

- 1. Serial.begin(9600);: Initializes serial communication with a baud rate of 9600 bits per second. This allows communication between the microcontroller and a connected computer for debugging purposes.
- 2. oled.begin();: Initializes the OLED display.
- 3. oled.clearDisplay();: Clears the OLED display to prepare for displaying new content.
- 4. oled.setTextSize(1);: Sets the text size on the OLED display to 1 (the default size).
- 5. oled.setTextColor(1);: Sets the text color on the OLED display. The parameter 1 likely represents white or another color, depending on the display's configuration.
- 6. oled.setCursor(0, 0);: Sets the cursor position on the OLED display to the top-left corner.
- 7. oled.println("Initializing pulse oximeter..");: Prints a message on the OLED display indicating that the pulse oximeter is being initialized.
- 8. oled.display();: Updates the OLED display to show the message.
- 9. Serial.print("Initializing pulse oximeter..");: Prints the same message to the serial monitor for debugging purposes.
- 10. if (!pox.begin()) { ... } else { ... }: Initializes the pulse oximeter. If initialization fails, it prints "FAILED" on both the serial monitor and the OLED display. If successful, it prints "SUCCESS" on both.
- 11. pox.setOnBeatDetectedCallback(onBeatDetected);: Sets a callback function onBeatDetected to be executed whenever a beat is detected by the pulse oximeter. •

```
void loop()
{
pox.update();

if (millis() - tsLastReport > REPORTING_PERIOD_MS) {
Serial.print("Heart BPM:");
Serial.print(pox.getHeartRate());
Serial.print("-----");
Serial.print("Oxygen Percent:");
Serial.print(pox.getSpO2());
Serial.println("\n");
oled.clearDisplay();
oled.setTextSize(1);
oled.setTextColor(1);
oled.setCursor(0,16);
oled.println(pox.getHeartRate());

oled.setTextSize(1);
oled.setTextColor(1);
oled.setCursor(0, 0);
oled.println("Heart BPM");
oled.setTextSize(1);
oled.setTextColor(1);
oled.setCursor(0, 30);
oled.println("Spo2");

oled.setTextSize(1);
oled.setTextColor(1);
oled.setCursor(0,45);
oled.println(pox.getSpO2());
oled.display();
tsLastReport = millis();
}
}
```

- The loop() function seems to be the main execution loop of the program. Here's what it does:
- 1. pox.update();: Updates the pulse oximeter readings. This function likely reads sensor data and updates internal variables with the latest measurements.
- 2. if (millis() - tsLastReport > REPORTING_PERIOD_MS) { ... }: Checks if it's time to report the pulse oximeter readings. This condition is true if the time elapsed since the last report (tsLastReport) is greater than the reporting period (REPORTING_PERIOD_MS). This condition ensures that readings are reported at regular intervals.
- 3. Inside the conditional block:
- - It prints the heart rate and oxygen saturation (SpO2) readings to the serial monitor.

- - It clears the OLED display to prepare for displaying new content.
- - It sets the text size, color, and cursor position on the OLED display.
- - It prints the heart rate and SpO2 readings on the OLED display.
- - It updates the OLED display to show the new readings.
- - It updates the tsLastReport variable to the current time, marking the time of the last report.
- 4. After the conditional block, the loop() function continues to iterate.